ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA KHOA HỌC MÁY TÍNH

BÀI TẬP 3

GIẢI GAME OF 24 SỬ DỤNG TREE OF THOUGHTS

Lớp: CS106.Q21
Giảng viên hướng dẫn: Lương Ngọc Hoàng

Họ và tên: Trần Mạnh Hùng
MSSV: 24520630

Ngày 20 tháng 4 năm 2026

Mục lục

1 Giới thiệu Game of 24 **2**

2 Mô hình hoá và tìm kiếm cổ điển **2**
2.1 Mô hình hoá 2
2.2 Tìm kiếm cổ điển 3

3 Tree of Thoughts với LLM **4**
3.1 Ý tưởng cốt lõi của Tree of Thoughts (ToT) 4
3.2 Thiết kế Prompt 4
3.3 Tối ưu hoá hệ thống và Xử lý ngoại lệ 5
3.4 Mã giả thuật toán ToT-DFS 5

4 Kết quả Thực nghiệm và Đánh giá **6**
4.1 Bảng kết quả Input-Output Prompting (Baseline) 6
4.2 Bảng kết quả Tree of Thoughts (ToT) 7
4.3 Nhận xét Input-Output Prompting (Phương pháp Baseline) 8
4.4 So sánh Tree of Thoughts (ToT) và Input-Output Prompting 9
4.5 Đánh giá tác động của kích thước Mô hình (70B vs 17B) 9

5 Kết luận **9**

1 Giới thiệu Game of 24

- Game of 24 là một trò chơi toán học đơn giản. Người chơi được cho 4 số nguyên. Nhiệm vụ là sử dụng các phép toán số học để kết hợp 4 số này sao cho kết quả cuối cùng bằng 24. Các phép toán được phép sử dụng là: + (cộng), - (trừ), * (nhân), / (chia). Người giải có thể dùng dấu ngoặc ( ) để thay đổi thứ tự tính toán.
- Mục tiêu của bài báo cáo này là giải quyết Game of 24 sử dụng thuật toán tìm kiếm cổ điển, cùng với đó là Tree of Thoughts với LLM.

2 Mô hình hoá và tìm kiếm cổ điển

2.1 Mô hình hoá

Để mô hình hoá Game of 24, ta cần xác định Không gian trạng thái bao gồm Trạng thái (State), Hành động (Action), Chuyển tiếp (Transition) và Mục tiêu (Goal).

- **Trạng thái:** Tập hợp các số đang có ở thời điểm hiện tại (các số có thể trùng nhau). Kí hiệu: S
  - Trạng thái bắt đầu (S_0): 4 số ban đầu.
  - Trạng thái trung gian (S_1 hoặc S_2): Ta gộp 2 số lại thành 1 số. Từ 4 số ban đầu giảm còn 3 số hoặc từ 3 số giảm còn 2 số.
  - Trạng thái kết thúc (S_3): Tập hợp chỉ còn 1 số.
- **Hành động:** Mỗi hành động sẽ có 2 bước:
  - Chọn 2 số bất kì (x và y) có trong trạng thái hiện tại.
  - Chọn toán tử bất kì ($+$, $-$, $*$, $/$) để thực hiện phép toán giữa x và y. Khi đó, ta có các hành động hợp lệ: $x+y$, $x-y$, $y-x$, $x*y$, x/y ($y \neq 0$), y/x ($x \neq 0$).
- **Chuyển tiếp:** Cập nhật trạng thái sau khi thực hiện một hành động hợp lệ.
  - Giả sử tại trạng thái S_i, ta chọn 1 toán tử $op \in \{+, -, *, /\}$ áp dụng lên 2 số x và y. Khi đó. Kết quả trả về $z = x\ op\ y$.
  - Trạng thái tiếp theo S_{i+1} được xác định bằng phép toán đa tập hợp (multiset) như sau:
    $$S_{i+1} = (S_i \setminus \{x, y\}) \cup \{z\}$$
- **Mục tiêu:** Trả về `True` nếu thoả mãn cả 2 điều kiện:
  - Tập hợp chỉ còn lại 1 số.
  - Giá trị của số còn lại là 24.

2.2 Tìm kiếm cổ điển

Ta sử dụng thuật toán tìm kiếm theo chiều rộng BFS (Breadth-First Search) để duyệt qua không gian trạng thái một cách hệ thống. Trong cài đặt thực tế, trạng thái của bài toán được biểu diễn bằng một danh sách các cặp giá trị và biểu thức tương ứng $\langle val, expr \rangle$. Thuật toán BFS sẽ vét cạn mọi tổ hợp cặp số khả dĩ ở mỗi bước cho đến khi biểu thức rút gọn chỉ còn một giá trị duy nhất.

Mã giả của thuật toán được trình bày chi tiết như sau:

Algorithm 1 Thuật toán BFS giải bài toán Game of 24 (Cài đặt Tuple)

```
procedure BFS(initial_state)
    Khởi tạo hàng đợi Q
    Q.push(initial_state)                ▷ Mỗi phần tử là mảng các tuple ⟨val, expr⟩
    while Q không rỗng do
        current_state ← Q.popleft()
        if |current_state| == 1 then                    ▷ Kiểm tra trạng thái đích
            ⟨val, expr⟩ ← current_state[0]
            if |val − 24| < ϵ then                          ▷ Kiểm tra xấp xỉ 24
                return expr                          ▷ Trả về chuỗi biểu thức kết quả
            end if
            continue
        end if
        n ← |current_state|
        for i ← 0 to n − 1 do                                   ▷ Chọn số thứ nhất
            for j ← 0 to n − 1 do                                   ▷ Chọn số thứ hai
                if i == j then continue
                end if
                ⟨val_1, expr_1⟩ ← current_state[i]
                ⟨val_2, expr_2⟩ ← current_state[j]
                next_state_base ← Lấy các phần tử không chứa vị trí i và j
                operations ← ∅
                operations.append(⟨val_1 + val_2, "(expr_1 + expr_2)"⟩)
                operations.append(⟨val_1 − val_2, "(expr_1 − expr_2)"⟩)
                operations.append(⟨val_1 ∗ val_2, "(expr_1 ∗ expr_2)"⟩)
                if val_2 ≠ 0 then                          ▷ Bảo vệ phép chia cho 0
                    operations.append(⟨val_1/val_2, "(expr_1/expr_2)"⟩)
                end if
                for mỗi ⟨next_val, next_expr⟩ ∈ operations do
                    next_state ← next_state_base ∪ {⟨next_val, next_expr⟩}
                    Q.push(next_state)        ▷ Thêm trạng thái mới vào cuối hàng đợi
                end for
            end for
        end for
    end while
    return Vô nghiệm
end procedure
```

Kết quả chạy chương trình được thống kê trong bảng sau:

Tập đầu vào (Test Case)	**Kết quả trả về**
$[4, 7, 8, 8]$	$8 \times ((4 + 7) - 8) = 24$
$[4, 9, 10, 13]$	$(4 - 10) \times (9 - 13) = 24$
$[1, 1, 1, 1]$	Không có lời giải
$[0, 9, 3, 3]$	$(3 \times (0 + 9)) - 3 = 24$
$[3, 3, 8, 8]$	$8 \div (3 - (8 \div 3)) = 24$
$[1, 2, 3, 4]$	$4 \times (3 + (1 + 2)) = 24$
$[6, 6, 6, 6]$	$(6 + 6) + (6 + 6) = 24$
$[2, 5, 6, 8]$	$8 \times (6 + (2 - 5)) = 24$
$[3, 5, 7, 9]$	$(3 + 5) + (7 + 9) = 24$
$[1, 5, 5, 5]$	$5 \times (5 - (1 \div 5)) = 24$

Bảng 1: Bảng thống kê kết quả chạy thực nghiệm thuật toán BFS

Nhận xét: Do không gian trạng thái của Game of 24 không quá lớn nên thuật toán không mất quá nhiều thời gian để chạy và đảm bảo tìm được kết quả nếu tồn tại.

3 Tree of Thoughts với LLM

3.1 Ý tưởng cốt lõi của Tree of Thoughts (ToT)

- Thay vì sử dụng phương pháp Input-Output Prompting truyền thống (yêu cầu LLM giải trực tiếp bài toán trong một lần gọi), phương pháp Tree of Thoughts (ToT) mô phỏng lại quá trình tư duy của con người. LLM sẽ được phân tác vụ để khám phá nhiều nhánh suy luận khác nhau, tự đánh giá mức độ khả thi của từng nhánh, và quay lui (backtrack) nếu đi vào ngõ cụt.

- Trong bài báo cáo này, kỹ thuật Tree of Thoughts (ToT) sẽ được kết hợp với thuật toán **DFS** để giải quyết bài toán suy luận phức tạp. Quá trình sinh và đánh giá trạng thái được thực hiện thông qua hai mô hình là **Llama 17B** và **Llama 70B**.

3.2 Thiết kế Prompt

Để LLM hoạt động chính xác như một hàm sinh (Generator) và hàm đánh giá (Evaluator) trong thuật toán tìm kiếm, ta có thể áp dụng kỹ thuật **Few-shot Prompting** kết hợp với các điều kiện về định dạng nghiêm ngặt:

- **Hàm Sinh ý tưởng (Generate):** Yêu cầu LLM sinh ra tối đa 5 phép toán hợp lệ từ tập số hiện tại. Để tránh LLM sinh ra văn bản thừa (hallucination), prompt được bổ sung các yêu cầu để LLM chỉ trả lời theo đúng định dạng, cùng với đó là vài ví dụ minh hoạ cụ thể.

- **Hàm Đánh giá (Evaluate):** Đóng vai trò là hàm Heuristic. LLM được yêu cầu phân loại trạng thái hiện tại thành đúng một trong ba từ khoá: `sure` (chắc chắn giải được), `likely` (có tiềm năng), hoặc `impossible` (vô nghiệm). Nhờ vậy, ta có thể cắt tỉa (prune) các nhánh vô ích một cách hiệu quả.

3.3 Tối ưu hoá hệ thống và Xử lý ngoại lệ

Trong quá trình cài đặt thực nghiệm với API của Groq, thuật toán DFS tạo nhánh liên tục dẫn đến các sự cố không mong muốn (Giới hạn request/phút, request/ngày, token/ngày,...). Để đảm bảo có thể chạy được thông suốt toàn bộ testcase, các kỹ thuật sau đã được áp dụng:

- **API Key Rotation (Xoay vòng khóa API):** Khởi tạo một mảng chứa nhiều API Keys từ các tài khoản khác nhau. Khi khoá API hiện tại đạt giới hạn, thuật toán tự động chuyển đổi sang khoá API khác để tiếp tục chạy mà không làm sập tiến trình.

- **Cơ chế Rate Limiting chủ động:** Sử dụng hàm `time.sleep()` chèn vào trước các lệnh gọi API để phân bổ đều lượng Request, tránh bị hệ thống máy chủ đánh dấu là Spam.

- **Bộ lọc Toán học:** Cài đặt khối `try-except` để bắt các lỗi logic do LLM ảo giác sinh ra (chia cho 0, output sai cú pháp,...), giúp cây DFS luôn duyệt trên các trạng thái hợp lệ.

3.4 Mã giả thuật toán ToT-DFS

Thuật toán được cài đặt theo chiến lược Tìm kiếm theo chiều sâu (DFS). Nhờ có cơ chế xoay vòng khóa API, thuật toán không cần sử dụng kỹ thuật cắt tỉa nhánh (Beam Search) mà sẽ duyệt qua toàn bộ các nhánh được LLM đánh giá là có tiềm năng (`sure` và `likely`), từ đó tăng tối đa khả năng tìm thấy lời giải.

Mã giả của thuật toán ToT-DFS được mô tả như sau:

Algorithm 2 Thuật toán Tree of Thoughts (ToT-DFS) giải Game of 24

```
procedure SOLVE24_TOT(initial_state)
    Khởi tạo ngăn xếp Stack ← [⟨initial_state, ∅⟩]
    Khởi tạo tập Visited ← ∅
    while Stack không rỗng do
        ⟨current_state, path⟩ ← Stack.pop()
        state_str ← Chuỗi biểu diễn đã sắp xếp của current_state
        if state_str ∈ Visited then continue
        end if
        Visited.add(state_str)
        if |current_state| == 1 then                                  ▷ Còn lại 1 số
            if |current_state[0] − 24| < ϵ then                       ▷ Kết quả xấp xỉ 24
                return path                                           ▷ Tìm thấy lời giải
            end if
            continue
        end if
        Thoughts ← LLM_GENERATE(current_state)        ▷ LLM sinh các ý tưởng
        ScoredSuccessors ← ∅
        for mỗi thought ∈ Thoughts do
            Giải mã thought thành next_state
            if Phép toán sai logic hoặc chia cho 0 then continue
            end if
            score ← LLM_EVALUATE(next_state)        ▷ LLM đánh giá: sure, likely, impossible
            if score ∈ {sure, likely} then
                Thêm ⟨score, thought, next_state⟩ vào ScoredSuccessors
            end if
        end for
        Sắp xếp ScoredSuccessors ưu tiên nhánh có điểm sure trước likely
            ▷ Đẩy TẤT CẢ nhánh tiềm năng vào Stack (không giới hạn Beam Width)
        for mỗi ⟨score, thought, next_state⟩ ∈ ScoredSuccessors (Duyệt ngược) do
            Stack.push(⟨next_state, path ∪ {thought}⟩)
        end for
    end while
    return Vô nghiệm
end procedure
```

4 Kết quả Thực nghiệm và Đánh giá

Để có cái nhìn trực quan về hiệu năng của các phương pháp, ta tiến hành tổng hợp kết quả chạy thực tế trên 10 bộ số (testcases) đại diện cho nhiều độ khó khác nhau.

4.1 Bảng kết quả Input-Output Prompting (Baseline)

Với phương pháp Baseline, LLM bị ép phải giải bài toán chỉ trong 1 lần suy luận. Kết quả cho thấy tỷ lệ sai sót là cực kỳ cao.

Test Case	Llama 70B	Llama 17B	Nhận xét
$[4, 7, 8, 8]$	$8 \times (7 - (8 \div 4))$	$8 \times (7 - (8 \div 8))$	70B sai kết quả ($40 \neq 24$). 17B thay đổi dữ liệu đầu vào
$[4, 9, 10, 13]$	$(13 - 9) \times (10 \div 4)$	$10+13+4-9\times4+13+10$	70B sai kết quả ($10 \neq 24$). 17B sai định dạng hoàn toàn
$[1, 1, 1, 1]$	impossible	impossible	Cả hai mô hình nhận diện đúng tập vô nghiệm
$[0, 9, 3, 3]$	$(3 \times 3) + (9 \div 0)$	impossible	70B mắc lỗi chia cho 0 17B bỏ cuộc (Sai).
$[3, 3, 8, 8]$	$(8 \times 3) + (8 \div 3)$	$8 \div (3 - 8 \div 3)$	70B sai kết quả ($64 \neq 24$). 17B lại cho kết quả đúng
$[1, 2, 3, 4]$	$(3 + 4 + 1) \times 2$	$(1 + 2) \times 3 \times 4$	Cả hai đều ra kết quả sai (16 và 36)
$[6, 6, 6, 6]$	$(6 \times 6) - (6 \div 6)$	$6 + 6 + 6 + 6$	70B sai kết quả ($40 \neq 24$). 17B đúng kết quả
$[2, 5, 6, 8]$	$(8 \times 3) + (5 + 2)$	$(8 - 5) \times (6 + 2)$	70B sai kết quả ($31 \neq 24$). 17B đúng kết quả
$[3, 5, 7, 9]$	$(9 \times 7) - (5 + 3)$	$(9 - 5) \times (7 - 3)$	Cả 2 đều ra kết quả sai (48 và 16)
$[1, 5, 5, 5]$	$(5 \times 5) - (5 \div 1)$	$5 \times (5 - (5 \div 5))$	70B sai kết quả ($20 \neq 24$). 17B thay đổi dữ liệu đầu vào

Bảng 2: Kết quả chạy thực nghiệm phương pháp Input-Output Prompting

4.2 Bảng kết quả Tree of Thoughts (ToT)

Khi áp dụng ToT, tỷ lệ giải thành công tăng lên rõ rệt. Tuy nhiên, vẫn có sự khác biệt lớn về hiệu năng giữa mô hình lớn (70B) và mô hình nhỏ (17B).

Bảng 3: So sánh kết quả chạy thực nghiệm bằng phương pháp Tree of Thoughts (ToT)

Test Case	**Llama 17B**	**Llama 70B**	**Đánh giá & Nhận xét**
$[4, 7, 8, 8]$	Không có lời giải	Tìm được đáp án	**70B thắng.** 70B tư duy tốt hơn ở các bước trừ cơ bản
$[4, 9, 10, 13]$	Tìm được đáp án	Tìm được đáp án	**Hòa.** Cả hai đều giải quyết tốt
$[1, 1, 1, 1]$	Không có lời giải	Không có lời giải	**Hòa.** (Đây là kết quả ĐÚNG vì tập này vô nghiệm)
$[0, 9, 3, 3]$	Không có lời giải	Tìm được đáp án	**70B thắng.** 17B bỏ cuộc quá sớm
$[3, 3, 8, 8]$	Không có lời giải	Không có lời giải	**Cùng thất bại.** Đây là câu hỏi khó yêu cầu phân số
$[1, 2, 3, 4]$	Tìm được đáp án	Tìm được đáp án (Lỗi)	**17B thắng.** 70B tìm ra đáp án nhưng lại thay đổi số 3 thành 4 ở bước đầu tiên
$[6, 6, 6, 6]$	Tìm được đáp án	Tìm được đáp án	**Hòa.** Câu cơ bản, xử lý tốt
$[2, 5, 6, 8]$	Tìm được đáp án	Không có lời giải	**17B bất ngờ thắng.** 70B có thể bị cắt tỉa nhầm nhánh đúng
$[3, 5, 7, 9]$	Tìm được đáp án (Lỗi)	Tìm được đáp án	**70B thắng.** 17B tìm được đáp án nhưng lại đổi số 5 thành 3 ở bước thứ 2
$[1, 5, 5, 5]$	Không có lời giải	Không có lời giải	**Cùng thất bại.** Tương tự với testcase $[3, 3, 8, 8]$

4.3 Nhận xét Input-Output Prompting (Phương pháp Baseline)

Khi yêu cầu các LLM giải quyết bài toán trực tiếp chỉ trong một lượt gọi (Zero-shot / Input-Output Prompting), cả hai mô hình đều bộc lộ điểm yếu chí mạng trong tư duy toán học nhiều bước.

- **Ảo giác toán học (Hallucination):** Các mô hình thường xuyên sinh ra các phép tính sai lệch hoàn toàn so với luật cơ bản. Ví dụ điển hình, mô hình Llama 70B với

tập đầu vào $[0, 9, 3, 3]$ đã sinh ra biểu thức $(3 \times 3) + (9 \div 0)$, vi phạm nghiêm trọng quy tắc chia cho 0.

- **Ảo giác dữ liệu:** Mô hình tự ý thay đổi các số ở đầu vào. Ví dụ, với tập $[2, 5, 6, 8]$, Llama 70B trả về $(8 \times 3) + (5 + 2)$, tự ý biến số 6 thành số 3. Mô hình 17B thậm chí còn mất kiểm soát định dạng ở tập $[4, 9, 10, 13]$ khi sinh ra chuỗi số dài lặp lại.

Ngoài ra, ở LLM kích thước nhỏ còn xuất hiện hiện tượng học vẹt kết quả. Cụ thể là Llama 17B đã trả lời đúng testcase khó $[3, 3, 8, 8]$ vì đây là một ví dụ rất điển hình của Game of 24, có thể model này đã được huấn luyện từ trước nên khi gặp lại, nó trả về kết quả rất nhanh và chính xác.

4.4 So sánh Tree of Thoughts (ToT) và Input-Output Prompting

Kết quả thực nghiệm cho thấy sự vượt trội tuyệt đối của cấu trúc ToT so với việc hỏi đáp trực tiếp.

- **Khả năng chia nhỏ vấn đề:** Thay vì bắt LLM tính nhẩm 3 phép toán cùng lúc, ToT ép LLM chỉ tính 1 phép toán ở mỗi bước sinh nhánh (Generate). Điều này loại bỏ hoàn toàn tình trạng tính toán sai lệch hay tự bịa ra số.

- **Tự sửa sai (Self-Correction):** Thông qua hàm đánh giá (Evaluate) tại mỗi nút, ToT cho phép thuật toán nhận diện được nhánh vô nghiệm để tiến hành quay lui (Backtrack), thay vì bị kẹt lại ở một biểu thức sai như phương pháp baseline.

4.5 Đánh giá tác động của kích thước Mô hình (70B vs 17B)

Trong cùng một khung thuật toán ToT, kích thước tham số của mô hình đóng vai trò quyết định đến độ chính xác cuối cùng:

- **Mô hình nhỏ (17B):** Có xu hướng đánh giá (Evaluate) kém chính xác. Nó thường xếp loại các nhánh khó (có chứa phân số trung gian) là `impossible`, khiến thuật toán DFS đi vào ngõ cụt và bỏ lỡ lời giải thực sự.

- **Mô hình lớn (70B):** Thể hiện xuất sắc trong việc làm trọng tài đánh giá (Heuristic). Năng lực ngôn ngữ và tư duy tốt giúp 70B nhìn ra được các tổ hợp phức tạp, dẫn dắt thuật toán tìm ra kết quả cuối cùng với tỷ lệ thành công cao hơn hẳn. Tuy nhiên, rào cản lớn nhất của 70B là chi phí Token cao và dễ gặp hiện tượng thắt cổ chai API (Rate Limit) do yêu cầu tài nguyên tính toán khổng lồ ở mỗi nhánh của ToT.

5 Kết luận

Cấu trúc Tree of Thoughts (ToT) kết hợp cùng thuật toán tìm kiếm DFS đã chứng minh được hiệu quả mạnh mẽ trong việc giải quyết bài toán Game of 24. ToT giúp khắc phục hoàn toàn điểm mù về tính toán logic của các LLM truyền thống. Dù chi phí vận hành (API Calls/Tokens) và thời gian thực thi (Latency) tăng lên đáng kể so với thuật toán vét cạn BFS cổ điển, ToT mở ra một hướng đi đầy tiềm năng để ứng dụng LLM vào các bài toán suy luận phức tạp trong thế giới thực.